Project Title: "Student Performance & Attendance Tracker"

Objective

Develop a **Student Performance & Attendance Tracker** using **MySQL**, where institutions can manage student records, monitor attendance, track performance across subjects, and generate reports. The system should support CRUD operations, filtering, sorting, aggregation, key relationships, joins, subqueries, and advanced SQL functions.

Problem Statement

Design a **Student Performance & Attendance Tracker** that enables educational institutions to:

1. **Manage Student & Faculty Information** (add, update, delete, search).
2. **Track Student Attendance** (daily logging, percentage calculation, and absence reports).
3. **Store and Analyze Student Performance** (marks, grades, subject-wise trends).
4. **Generate Insights & Reports** (top-performing students, attendance defaulters).

Database Schema

Tables & Relationships

1. **Students**
    - `student_id` (Primary Key)
    - `name`
    - `dob`
    - `gender`
    - `email`
    - `phone_number`
    - `address`

- admission_date
- department_id (Foreign Key)

2. **Faculty**
   - faculty_id (Primary Key)
   - name
   - email
   - phone_number
   - department_id (Foreign Key)
3. **Departments**
   - department_id (Primary Key)
   - department_name
4. **Courses**
   - course_id (Primary Key)
   - course_name
   - faculty_id (Foreign Key referencing Faculty.faculty_id)
5. **Enrollments**
   - enrollment_id (Primary Key)
   - student_id (Foreign Key referencing Students.student_id)
   - course_id (Foreign Key referencing Courses.course_id)
   - enrollment_date
6. **Attendance**
   - attendance_id (Primary Key)
   - student_id (Foreign Key referencing Students.student_id)
   - course_id (Foreign Key referencing Courses.course_id)
   - attendance_date
   - status (Present, Absent, Late)
7. **Grades**
   - grade_id (Primary Key)
   - student_id (Foreign Key referencing Students.student_id)
   - course_id (Foreign Key referencing Courses.course_id)
   - marks_obtained

- `grade`

Tasks & Functionalities

1. Implement CRUD Operations (Low Weightage)

- Insert new students, faculty members, courses, and enrollments.
- Update student records when they change contact details.
- Delete students who have dropped out.

2. Use SQL Clauses (WHERE, HAVING, LIMIT) (Low Weightage)

- Get students enrolled in the **Computer Science Department**.
- Retrieve the top **10 highest-scoring students**.
- Find students with **attendance below 75%**.

3. Apply SQL Operators (AND, OR, NOT) (Medium Weightage)

- Retrieve students who have attendance **below 50% AND are failing**.
- Find students who scored **above 90 OR have perfect attendance**.
- List faculty members **NOT assigned to any course**.

4. Sorting & Grouping Data (ORDER BY, GROUP BY) (Medium Weightage)

- List students **alphabetically by name**.
- Count the number of students enrolled in each department.
- Show the **average marks** per course.

5. Use Aggregate Functions (SUM, AVG, MAX, MIN, COUNT) (High Weightage)

- Find the **average attendance percentage** of students.
- Identify the **highest and lowest marks obtained** in each course.
- Calculate the **total number of students per department**.

6. Establish Primary & Foreign Key Relationships (High Weightage)

- Ensure that students cannot enroll in the same course multiple times.
- Link faculty members to the courses they teach.

7. Implement Joins (High Weightage)

- Retrieve student details along with their department using **INNER JOIN**.
- Get a list of **students who have not enrolled in any course** using **LEFT JOIN**.

- List courses that have no faculty assigned using **RIGHT JOIN**.
- Show students **without grades** using **FULL OUTER JOIN**.

8. Use Subqueries (High Weightage)

- Find students with marks **above the average score**.
- Retrieve courses taught by faculty members **with at least 5 years of experience**.
- Identify students who have **missed more than 10 classes**.

9. Implement Date & Time Functions (High Weightage)

- Extract the **month from attendance_date** to analyze attendance trends.
- Calculate the **number of years since a student's admission**.
- Format `attendance_date` as **DD-MM-YYYY**.

10. Use String Manipulation Functions (High Weightage)

- Convert all faculty names to **uppercase**.
- Trim unnecessary spaces from student names.
- Replace `NULL` email fields with `"Email Not Provided"`.

11. Implement Window Functions (Very High Weightage)

- Rank students based on their **overall marks**.
- Show the **cumulative attendance percentage** per course.
- Display the **running total of students** enrolled per month.

12. Apply SQL CASE Expressions (Very High Weightage)

- Assign **student performance levels**:
  - `"Excellent"` if `marks_obtained > 90`.
  - `"Good"` if `marks_obtained BETWEEN 75 AND 90`.
  - `"Needs Improvement"` otherwise.
- Categorize **attendance records**:
  - `"Regular"` for attendance above 80%.
  - `"Irregular"` for attendance between 50% and 80%.
  - `"Defaulter"` otherwise.

Instructions:

1. Write clean, modular, and well-commented code.